ReviveAI — AI Revenue Recovery Agent

Razorpay Buildathon · Track 03: AI Revenue Recovery

Full Project Plan: Design → Roadmap → Replit Prompts → Judge Critique

---

PART 1 — HOW THE PRODUCT WORKS (Three Perspectives)

1. The Merchant's Perspective

A merchant (say, a D2C brand or SaaS company using Razorpay) is bleeding revenue silently in four places:

- A subscription auto-debit fails (card expired, insufficient balance, bank declines the mandate)
- A customer starts checkout, enters payment details, and abandons before paying
- A one-time payment fails (wrong OTP, network timeout, bank down)
- An invoice (B2B) goes unpaid past its due date

Today, the merchant either does nothing (money is just lost) or manually chases each case — which doesn't scale past a few dozen customers.

What the merchant experiences with ReviveAI:

1. They connect/upload their transaction and subscription data (in the MVP: CSV upload or synthetic Razorpay-shaped data, framed as "this is what a Razorpay webhook feed would look like").
2. They open a dashboard and see: "₹4,82,300 at risk this week across 217 events."
3. For each at-risk event, they see: **what happened, why it likely happened (AI diagnosis), what ReviveAI recommends doing, and whether it's allowed to happen automatically** (per merchant-configured policy).
4. They see a live **Recovery Queue** — actions taken, actions pending approval, actions blocked by safety rules.
5. At the end of a batch run, they see a **recovered revenue number** with proof: which specific transactions moved from "at risk" to "recovered," and the full reasoning trail behind each one.

6. Crucially: **the merchant is always in control.** They set the rules (e.g., "max 2 reminders, no discount above 10%, never contact a customer more than once per 48 hours, always require human approval above ₹10,000"). The AI never overrides these.

This is the emotional core of the pitch: *"Don't just tell me I'm losing money — go get it back, but never do anything I wouldn't approve of."*

2. The AI System's Perspective

The system is a **pipeline of five deterministic stages**, where the LLM only participates in one stage (Diagnosis) as an advisor — never as the executor. This is the single most important architectural decision for this hackathon, because it directly answers Razorpay's "compliant escalation, stopping rules" bar and is exactly what fintech judges are trained to scrutinize (LLMs must not have unilateral authority over money-movement or customer-contact actions).

[1] Detection Engine (deterministic, rule/stat-based)

↓ produces a "RiskEvent"

[2] Diagnosis Agent (LLM — Gemini)

↓ produces a "Diagnosis" (root cause + suggested action + confidence)

[3] Policy Engine (deterministic, pure Python rules)

↓ produces a "Decision" (approve / modify / block / escalate to human)

[4] Action Executor (deterministic, simulated side effects)

↓ produces an "ActionResult"

[5] Audit Logger (deterministic, append-only)

↓ writes immutable "AuditLog" row for every single step above

Every RiskEvent flows through all five stages, and every stage's input/output is logged. The LLM's output is treated as **untrusted advisory text** — the Policy Engine parses it, validates it against hard rules, and can override or block it entirely. This means:

- If the LLM recommends a 30% discount but policy caps discounts at 10%, the Policy Engine clips it to 10% and logs the override.
- If the LLM recommends contacting a customer who was already contacted twice this week, the Policy Engine blocks the action and logs why.

- If the LLM is uncertain (low confidence) or the amount is large, the Policy Engine escalates to "needs human approval" instead of auto-executing.

This gives you a defensible, explainable, "AI recommends, system decides" architecture — which is both genuinely good engineering and a strong story for judges.

Trust Boundary Matrix

Put this table directly in your README — it's the fastest way to communicate the entire safety architecture to a judge in under 15 seconds:

Component	Role	Execution Authority
LLM (Gemini/Groq)	Diagnoses root cause, proposes an action	**None.** Can only output a typed JSON diagnosis; cannot write to the DB, cannot trigger a message, cannot touch money.
Policy Engine	Gating & decisioning	**Absolute.** Overrides, clips, blocks, or escalates any LLM proposal against hard business rules.
State Store (SQLite)	Idempotency & persistence	**Absolute.** A `PENDING` lock + unique constraint physically prevents the same risk_event from being executed twice, even under a bug or concurrent trigger.
Action Executor	Physical dispatch (simulated)	**Execution only.** Dispatches exactly what the Policy Engine decided — cannot deviate from it.
Streamlit UI / Human	Operator console	**Read + Approve/Reject only.** Cannot propose actions itself; can only approve or block what the pipeline already produced.

3. The Judge's Perspective

A judge evaluating 40+ submissions in this track will mentally run this checklist in under 5 minutes:

- **Does it detect real signal, or is it a wrapper around ChatGPT?** → You need a real Detection Engine with actual logic (see Part 2).
- **Does the AI have unchecked authority over money/customer contact?** → If yes, instant red flag for a payments company. Your Policy Engine is the answer.
- **Is there a number at the end?** → "We recovered ₹X out of ₹Y at risk (Z% recovery rate)" is the single most important sentence in your demo.
- **Is there an audit trail?** → Fintech = compliance. A judge from Razorpay will specifically look for this because it's in the brief.
- **Are there stopping rules?** → Does the system know when to stop bothering a customer? (max attempts, cooldown periods, opt-out handling)
- **Is it demoable in 3 minutes?** → Batch mode with a "Run Recovery Batch" button that visibly processes N events and shows before/after revenue is far more demoable than a live-only system.

What makes a project win this track specifically: it's not the LLM call (everyone will have one). It's the **policy engine + audit trail + measured outcome** — because that's the part that proves you understood *why* this is a fintech problem, not just an AI problem.

PART 2 — COMPLETE MVP DESIGN

2.1 User Flow

1. Merchant logs in (single demo merchant account is fine for MVP)

2. Merchant lands on Overview Dashboard

   → sees total revenue at risk, broken down by category

3. Merchant clicks "Run Detection" (or it's scheduled/automatic on data load)

   → Detection Engine scans transaction/subscription/checkout/invoice tables

   → creates RiskEvent rows

4. Merchant clicks "Diagnose & Recommend" (or auto-triggers per event)

→ Diagnosis Agent (Gemini) analyzes each RiskEvent

→ produces root cause + recommended action + confidence score

5. Merchant clicks "Run Recovery Batch"

→ Policy Engine evaluates every diagnosed event against rules

→ Auto-approved actions → Action Executor → simulated execution

→ Blocked/escalated actions → sent to "Needs Approval" queue

6. Merchant reviews "Needs Approval" queue, approves/rejects manually

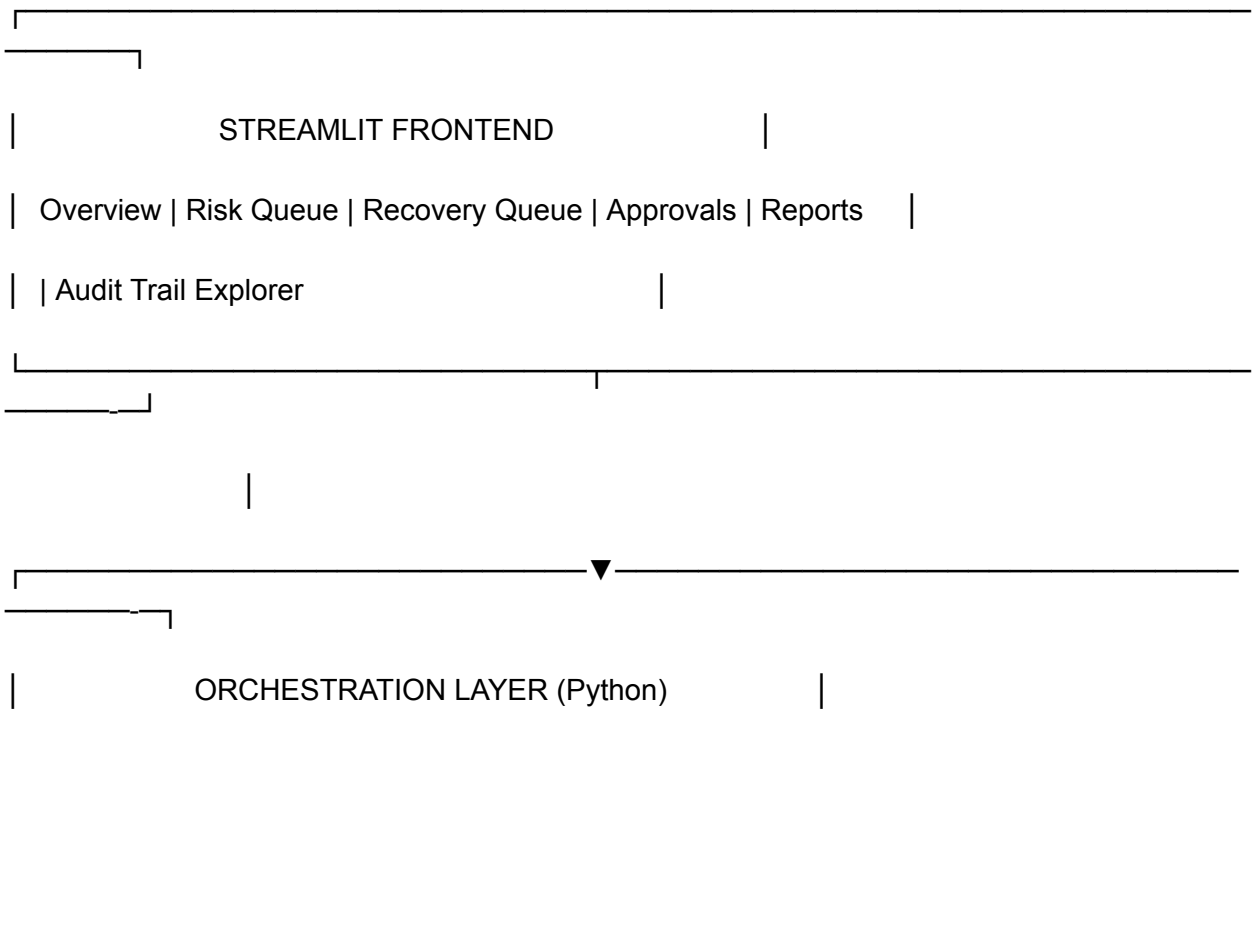
7. Merchant watches "Outcome Tracker"

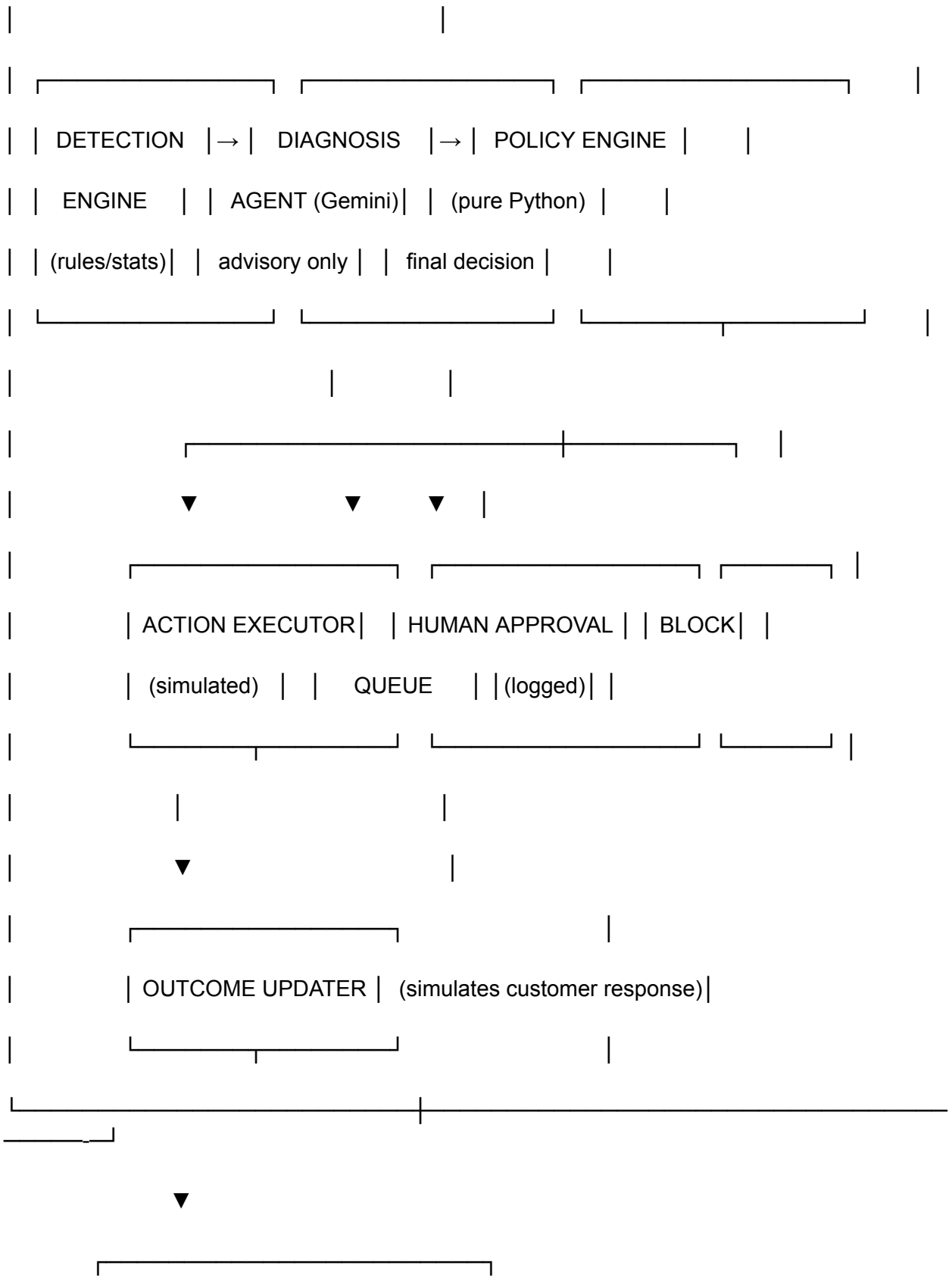
→ simulated customer responses (paid / ignored / opted out) update over time

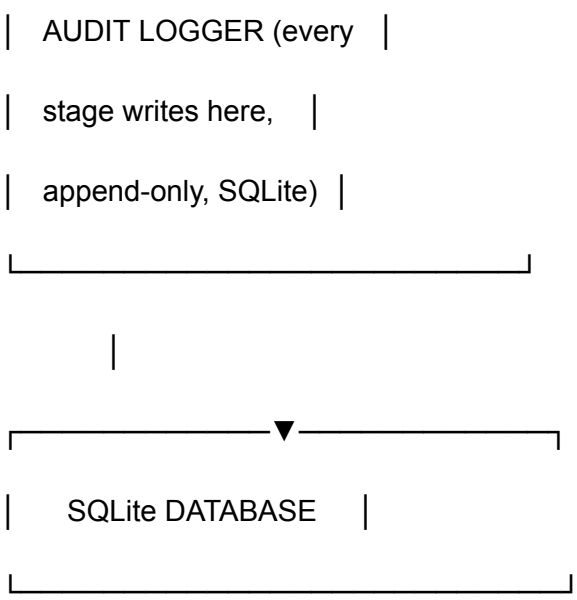
8. Merchant opens Reports screen

→ sees ₹ recovered, recovery rate, breakdown by risk type, audit trail export

2.2 System Architecture




DETECTION ENGINE (rules/stats)
DIAGNOSIS AGENT (Gemini) advisory only
POLICY ENGINE (pure Python) final decision
ACTION EXECUTOR (simulated)
HUMAN APPROVAL QUEUE
BLOCK (logged)
OUTCOME UPDATER (simulates customer response)



Key principle reflected in the architecture: the Gemini API is called from exactly ONE module (`diagnosis_agent.py`). It never touches the database, never triggers an action, never has execution authority. This isolation is easy to point to in your code walkthrough and is a strong signal of good engineering judgment.

2.3 Database Schema (SQLite)

```
-- Core entity: who we're recovering money from

CREATE TABLE customers (
    customer_id TEXT PRIMARY KEY,
    name TEXT,
    email TEXT,
    phone TEXT,
    preferred_language TEXT DEFAULT 'en',   -- for Hinglish angle
    contact_count_last_7d INTEGER DEFAULT 0,
    opted_out INTEGER DEFAULT 0,            -- hard stop flag
    created_at TIMESTAMP
```

```
);

-- Raw revenue events (simulates Razorpay webhook data)
CREATE TABLE revenue_events (
    event_id TEXT PRIMARY KEY,
    customer_id TEXT REFERENCES customers(customer_id),
    event_type TEXT,        -- 'payment_failed' | 'checkout_abandoned' |
                            -- 'subscription_failed' | 'invoice_overdue'
    amount REAL,
    currency TEXT DEFAULT 'INR',
    payment_method TEXT,    -- 'card' | 'upi' | 'netbanking' | 'mandate'
    failure_code TEXT,      -- raw gateway/bank decline code, if any
    subscription_id TEXT,
    invoice_id TEXT,
    checkout_session_id TEXT,
    occurred_at TIMESTAMP,
    raw_metadata TEXT       -- JSON blob, mimics Razorpay payload
);

-- Output of Detection Engine
CREATE TABLE risk_events (
    risk_id TEXT PRIMARY KEY,
    event_id TEXT UNIQUE REFERENCES revenue_events(event_id),   -- UNIQUE =
idempotency guard #1:
```

```
                                        -- one risk_event per source event, ever
    customer_id TEXT REFERENCES customers(customer_id),
    risk_category TEXT,     -- 'payment_degradation' | 'checkout_dropoff' |
                            -- 'subscription_failure' | 'receivable_overdue'
    risk_score REAL,        -- 0-1, from detection logic
    amount_at_risk REAL,
    detected_at TIMESTAMP,
    status TEXT DEFAULT 'new',   -- new | locked | diagnosed | actioned | resolved | expired | rejected_by_merchant
    lock_token TEXT,             -- idempotency guard #2: set atomically before execution begins
    locked_at TIMESTAMP          -- if set and stale (>N minutes) without a resulting action, the lock
                                 -- is considered abandoned and is swept back to 'diagnosed'/STOP_AND_ESCALATE
);
-- Output of Diagnosis Agent (LLM) — advisory only
CREATE TABLE diagnoses (
    diagnosis_id TEXT PRIMARY KEY,
    risk_id TEXT REFERENCES risk_events(risk_id),
    root_cause TEXT,             -- LLM's explanation
    recommended_action TEXT,     -- 'send_reminder' | 'retry_payment' |
                                 -- 'offer_discount' | 'send_payment_link' |
                                 -- 'escalate_to_human' | 'switch_payment_method'
```

```
    recommended_channel TEXT,   -- 'email' | 'sms' | 'whatsapp'
    recommended_discount_pct REAL DEFAULT 0,
    confidence REAL,            -- 0-1
    llm_raw_response TEXT,      -- full response, stored for audit
    diagnosed_at TIMESTAMP
);
-- Output of Policy Engine — the FINAL decision, always deterministic
CREATE TABLE policy_decisions (
    decision_id TEXT PRIMARY KEY,
    risk_id TEXT REFERENCES risk_events(risk_id),
    diagnosis_id TEXT REFERENCES diagnoses(diagnosis_id),
    decision TEXT,              -- 'auto_approved' | 'modified' | 'blocked' | 'needs_human_approval'
    final_action TEXT,          -- what will actually execute (may differ from LLM's suggestion)
    final_discount_pct REAL,
    rules_applied TEXT,         -- JSON list of which rules fired, e.g.
["discount_cap","cooldown_check"]
    reason TEXT,                -- human-readable explanation of the decision
    decided_at TIMESTAMP
);
-- Output of Action Executor
CREATE TABLE actions_taken (
    action_id TEXT PRIMARY KEY,
```

```
    decision_id TEXT UNIQUE REFERENCES policy_decisions(decision_id),  -- idempotency guard #3:
                                                                       -- one execution per decision, enforced
                                                                       -- by the DB, not application logic
    risk_id TEXT REFERENCES risk_events(risk_id),
    action_type TEXT,
    channel TEXT,
    message_sent TEXT,          -- simulated message content
    executed_at TIMESTAMP,
    simulated INTEGER DEFAULT 1  -- always 1 in MVP — be explicit about this
);
-- Outcome tracking (simulated customer behavior over time)
CREATE TABLE outcomes (
    outcome_id TEXT PRIMARY KEY,
    action_id TEXT REFERENCES actions_taken(action_id),
    risk_id TEXT REFERENCES risk_events(risk_id),
    outcome_type TEXT,      -- 'paid' | 'ignored' | 'opted_out' | 'failed_again'
    amount_recovered REAL DEFAULT 0,
    occurred_at TIMESTAMP
);
-- The append-only audit trail — every single stage writes here
CREATE TABLE audit_log (
```

```
    log_id TEXT PRIMARY KEY,
    risk_id TEXT,
    stage TEXT,              -- 'detection' | 'diagnosis' | 'policy' | 'execution' | 'outcome'
    actor TEXT,              -- 'system' | 'gemini_llm' | 'policy_engine' | 'human:<user_id>'
    input_snapshot TEXT,     -- JSON of what went in
    output_snapshot TEXT,    -- JSON of what came out
    timestamp TIMESTAMP
);
-- Merchant-configurable policy rules (drives Policy Engine)
CREATE TABLE policy_rules (
    rule_id TEXT PRIMARY KEY,
    rule_name TEXT,
    rule_type TEXT,          -- 'max_contact_frequency' | 'discount_cap' |
                             -- 'approval_threshold_amount' | 'cooldown_hours' |
                             -- 'max_attempts' | 'opt_out_respect'
    rule_value TEXT,         -- JSON, e.g. {"max_discount_pct": 10}
    active INTEGER DEFAULT 1
);
```

This schema is deliberately "boringly correct" — every table maps 1:1 to a pipeline stage, which makes your architecture diagram and your database self-documenting to a judge.

2.4 Dashboard Screens (Streamlit)

1. **Overview** — headline metrics: total revenue at risk, total recovered, recovery rate %, breakdown by risk category (pie/bar chart), trend over the batch run.
2. **Risk Queue** — table of all `risk_events`, filterable by category/status, click into any row to see full detail.
3. **Diagnosis View** — for a selected risk event: root cause, recommended action, confidence, LLM's raw reasoning (shown transparently — "here's exactly what the AI said").
4. **Recovery Queue** — shows Policy Engine decisions: auto-approved / modified / blocked / needs approval, with the `reason` field visible for every single one.
5. **Approvals Inbox** — human-in-the-loop screen: merchant reviews escalated cases, clicks Approve/Reject, action executes only after this.
6. **Outcomes & Reports** — the money slide: ₹ recovered vs ₹ at risk, recovery rate by category, time-to-recovery, exportable CSV.
7. **Audit Trail Explorer** — search/filter the `audit_log` table by risk_id; this is the screen you open when a judge asks "how do I know the AI didn't just do whatever it wanted?"

2.5 Metrics to Track

- **Total Revenue at Risk (₹)** — sum of `amount_at_risk` for open risk_events
- **Total Revenue Recovered (₹)** — sum of `amount_recovered` from outcomes
- **Recovery Rate (%)** — recovered / at-risk
- **Recovery Rate by Category** — subscription vs checkout vs payment vs invoice
- **Auto-Approved vs Human-Approved vs Blocked** — proportion of decisions in each bucket (this is your "safety" metric — show this prominently)
- **Average Time to Recovery**
- **Contact Efficiency** — recovered ₹ per message sent (proves you're not just spamming customers)
- **Opt-out / stopping-rule triggers** — how many times the system correctly stopped itself

2.6 Recovery Workflows (per category)

Subscription payment failure: `detect (mandate/auto-debit declined)` → `diagnose (soft decline vs hard decline vs expired card)` → `policy (if soft decline: auto-retry with backoff; if hard decline: send update-payment-method link; cap retries at 3, cooldown 24h between retries)` → `execute (simulated retry / simulated link send)` → `track (paid / still failing / churned)`

Checkout abandonment: `detect (session started, no payment within N minutes)` → `diagnose (price hesitation vs payment method friction vs technical failure)` → `policy (send reminder within 1h, max 2 reminders`

`24h apart, discount only if cart value under merchant-set threshold and never over cap)` → `execute (simulated email/SMS with recovery link)` → `track`

Payment failure (one-off): `detect (gateway decline)` → `diagnose (bank down, insufficient funds, wrong OTP, network)` → `policy (if transient: auto-retry once immediately; if hard decline: suggest alternate payment method)` → `execute` → `track`

B2B receivables (overdue invoice): `detect (invoice past due_date)` → `diagnose (first-time late vs chronic late-payer, amount size)` → `policy (staged escalation: friendly reminder → firmer reminder → offer payment plan → escalate to human/collections; NEVER auto-escalate to legal/collections without human approval)` → `execute` → `track (promise-to-pay date logged, followed up automatically)`

This last one — **Promise-to-Pay Tracker** — is a great differentiator: when a customer replies "I'll pay by Friday," the system logs a `promise_to_pay` state with a date, and auto-checks on that date, escalating only if broken. This is a realistic, judge-legible feature that's cheap to build (just a status + date field + a scheduled check).

2.7 Safety Rules (the Policy Engine's actual logic)

These are the rules Razorpay judges will specifically look for. Implement these as literal, readable Python functions — not LLM calls:

1. **Frequency cap** — no customer contacted more than 2x in 7 days, regardless of what the LLM recommends.
2. **Cooldown period** — minimum 24h between consecutive actions on the same risk event.
3. **Max attempts** — after 3 failed recovery attempts, auto-stop and mark `expired` — no infinite chasing.
4. **Discount cap** — hard ceiling (e.g. 10%) that the Policy Engine enforces regardless of LLM suggestion.
5. **Approval threshold** — any action on amount > ₹10,000 (configurable) requires human approval, never auto-executes.
6. **Opt-out respect** — if `customers.opted_out = 1`, the Policy Engine blocks ALL actions unconditionally — this check runs first, before anything else.
7. **Confidence threshold** — if LLM diagnosis confidence < 0.6, auto-escalate to human review instead of auto-executing.
8. **Channel restriction** — no WhatsApp/SMS outside configured hours (e.g. 9am–8pm) — a real compliance-style rule (this maps to actual DLT/TRAI-style constraints in India, which is a nice authentic touch).

9. **Duplicate suppression** — never send two active actions for the same risk_event simultaneously.
10. **At-most-once execution (idempotency)** — before the LLM is even called, the risk_event must be atomically moved from `status='diagnosed'` to `status='locked'` with a `lock_token` written in the same transaction. If two triggers fire on the same risk_event concurrently (e.g. a duplicate webhook, a double-click in the UI, a retried batch job), only one wins the lock — the DB's `UNIQUE`/atomic-update constraint rejects the second, not application-level "if" checks. If a lock is acquired but the pipeline crashes before an action is recorded (a stale lock, `locked_at` older than e.g. 10 minutes with no matching row in `actions_taken`), a sweep step resets it to `status='diagnosed'` and forces the next attempt through **STOP_AND_ESCALATE** (human review) rather than silently retrying — a system should never guess its way out of an unknown crash state involving money.

Every one of these should produce a `rules_applied` entry and a human-readable `reason` in `policy_decisions` — that's what makes the system "explainable."

2.8 Audit Log Structure

Already defined in schema (2.3) — but the presentation matters. In the Audit Trail Explorer screen, render each risk_event's full lifecycle as a **timeline**, e.g.:

[10:02:14] DETECTION → system → risk_event created: subscription_failed, ₹1,499, score 0.82

[10:02:15] DIAGNOSIS → gemini_llm → root_cause: "card likely expired", action: send_update_link, confidence: 0.78

[10:02:15] POLICY → policy_engine → decision: auto_approved, reason: "confidence>0.6, amount<10000, no cooldown conflict"

[10:02:16] EXECUTION → system → simulated email sent via 'update_payment_method' template

[14:30:02] OUTCOME → system → customer updated card, payment succeeded, ₹1,499 recovered

This timeline view, more than any chart, is what will make a judge say "okay, this team actually thought about compliance."

PART 3 — DEVELOPMENT ROADMAP (Phased for Replit, ~10 days)

(Phase 2.5 — idempotency locking — adds roughly half a day; if you're tight on time, Phases 6 (Hinglish/Promise-to-Pay) is the one to compress or cut, not this one.)

General approach: build and test each stage of the pipeline in isolation before wiring them together, and keep every Replit prompt scoped to ONE deliverable so you can verify it works before moving on.

Phase 0 — Project Skeleton & Synthetic Data (Day 1)

Goal: Set up the repo structure, SQLite schema, and a realistic synthetic dataset that mimics Razorpay-shaped events.

Features:

- Folder structure: `/db`, `/detection`, `/diagnosis`, `/policy`, `/executor`, `/audit`, `/app.py` (Streamlit entry)
- SQLite DB initialized with the full schema from 2.3
- A data generator script producing ~150–300 synthetic customers and revenue_events across all 4 categories, with realistic distributions (e.g. most checkout abandonments cluster near cart values ₹500–5000; some customers are repeat late-payers, etc.)

Expected Output: A working `revive.db` file with realistic seed data you can query and eyeball for sanity.

Testing Strategy: Run `sqlite3 revive.db` (or a quick Python script) and manually check: do the amounts look realistic? Are categories distributed as expected? Is there at least one clearly "chronic late payer" customer with multiple overdue invoices (useful for demo)?

Phase 1 — Detection Engine (Day 2)

Goal: Deterministic logic that scans `revenue_events` and produces `risk_events` with a risk_score — no LLM involved.

Features:

- Rule-based detectors per category (e.g., checkout abandoned if session started but no payment_success event within 30 min; subscription failure if 2+ consecutive failed debits; invoice overdue if `due_date < today`)
- A simple risk_score formula (e.g., weighted by amount, days overdue/since failure, and customer's historical pay behavior)
- Writes to `risk_events` and logs a `detection` row to `audit_log` for each

Expected Output: `risk_events` table populated, printed summary showing total ₹ at risk.

Testing Strategy: Spot-check 5 risk_events against their source revenue_events manually — does the risk_score make sense? Re-run the script and confirm it doesn't create duplicate risk_events for the same source event (add a check).

Phase 2 — Diagnosis Agent, Provider-Agnostic (Gemini + Groq) (Day 3)

Goal: Wire up the LLM as an isolated, advisory-only module — but build it against a **provider-agnostic interface** so you can flip between Gemini and Groq with a single environment variable, decide which to actually demo on once you've tested both, and even fail over automatically if one is unavailable during your live demo. This is the ONLY phase that touches an LLM API.

Why build it this way: you don't yet know which provider will be reliable for you (API access, rate limits, latency on demo day). Hardcoding one now means a rewrite later if it doesn't work out. An adapter pattern costs you ~20 extra minutes today and removes that risk entirely — and "we built a provider-agnostic LLM layer so we're not locked into a single vendor" is also a legitimate, judge-worthy engineering point on its own.

Features:

- A single abstract interface `diagnose_risk_event(risk_event_row) -> dict` that the rest of the pipeline calls — it never knows or cares which provider actually ran.
- Two concrete adapters: `GeminiProvider` and `GroqProvider`, both implementing the same method signature and returning the same normalized dict shape.
- Provider selection via an environment variable `LLM_PROVIDER` (`"gemini"` or `"groq"`), read once at startup — no code changes needed to switch, just change the env var and rerun.
- **Automatic fallback:** if the primary provider errors out (auth failure, rate limit, timeout) mid-batch, the adapter catches it, logs the failure to `audit_log`, and retries the same risk_event on the secondary provider before giving up and falling back to

`escalate_to_human`. This means your live demo can't be taken down by one provider having a bad day.
- Both providers are instructed to return **identical JSON schema**, so `diagnoses` rows never need a "which provider produced this" branch anywhere except a `provider_used` column for transparency/audit.

Expected Output: `diagnoses` table populated with LLM reasoning for each risk_event, each tagged with `provider_used`; printed log showing which provider handled each call and how many needed fallback.

Testing Strategy:

- Set `LLM_PROVIDER=groq`, run the batch, confirm it works end-to-end and check latency (this is likely your demo-day choice given free-tier speed).
- Set `LLM_PROVIDER=gemini`, run the same batch, compare quality of `root_cause` explanations and latency between the two — pick whichever is more reliable for you closer to submission day.
- Deliberately break one provider (feed a bad API key) and confirm the pipeline automatically falls through to the other provider without crashing, and that the fallback is visible in `audit_log`.
- Manually read 10 diagnoses regardless of provider — do the root causes make sense given the failure_code?
- Test the total-failure case (both keys invalid) and confirm it degrades gracefully to `escalate_to_human` rather than crashing the batch.

Phase 2.5 — Idempotency Lock + Failure Injection Tests (Day 4, short but high-value)

Goal: Guarantee at-most-once execution at the database level — not with application-level "if already done" checks, which can race — and prove it with adversarial tests. This is the single most convincing "real fintech engineering" moment you can add, and it's cheap.

Features:

- `acquire_lock(risk_id)` — an atomic SQL update (`UPDATE risk_events SET status='locked', lock_token=?, locked_at=? WHERE risk_id=? AND status='diagnosed'`) — checked via `cursor.rowcount`. If `rowcount == 0`, someone else already has the lock; the caller must back off, not proceed.
- `sweep_stale_locks()` — finds any `risk_events` with `status='locked'` and `locked_at` older than N minutes with no matching row in `actions_taken`, resets them to `status='diagnosed'`, and forces their next diagnosis to auto-map to

`STOP_AND_ESCALATE` (human review) rather than silently retrying — logs this as `stage='lock_swept'`.

- Three adversarial test scripts, directly modeled on real concurrency bugs:
  - **Concurrent duplicate trigger** — fire the pipeline on the *same* risk_id from two threads/processes at once; assert exactly one succeeds and the other is cleanly rejected by the lock.
  - **Stale/abandoned lock** — manually set a `locked_at` timestamp far in the past with no resulting action, run the sweep, assert it resets to `diagnosed` and the next pass escalates to human rather than auto-executing.
  - **Duplicate executor call** — call `execute_action()` twice for the same `decision_id`; assert the second call is rejected by the `UNIQUE` constraint on `actions_taken.decision_id`, not by an `if` check in Python.

Expected Output: Three passing adversarial tests you can show live or reference in your pitch as proof of at-most-once execution.

Testing Strategy: Actually run all three tests and watch them pass before moving on — this phase is entirely about proving a safety property, so don't skip verifying it. For your demo, consider live-triggering the same risk_event twice in the Streamlit UI (double-click a "Run Recovery" button) and showing the second click gets cleanly rejected with a visible message, not a crash or a duplicate action.

Phase 3 — Policy Engine (Day 4–5, most important phase)

Goal: The deterministic decision-maker. This is the phase judges will scrutinize most — spend real time here.

Features:

- `policy_rules` table seeded with sensible defaults
- A function `evaluate(risk_event, diagnosis) -> Decision` that runs every safety rule from section 2.7 in order, short-circuiting on the first hard block (e.g., opted_out)
- Produces `final_action`, `final_discount_pct` (clipped to policy caps), `decision` (auto_approved / modified / blocked / needs_human_approval), and a human-readable `reason` string listing every rule that fired
- Writes to `policy_decisions` and logs a `policy` row to `audit_log`

Expected Output: `policy_decisions` populated; printed breakdown (this is your "safety" metric for the demo — e.g. "68% auto-approved, 12% modified, 15% escalated to human, 5% blocked").

Testing Strategy: This phase needs the most manual testing. Deliberately construct edge-case rows (an opted-out customer with a diagnosis, a ₹50,000 risk event, a low-confidence diagnosis) and confirm each hits the correct rule. This is exactly what you should screen-record for your demo video as a "look, the AI wanted to do X, but the policy engine blocked/modified it because Y" moment.

Phase 4 — Action Executor + Outcome Simulator (Day 6)

Goal: Simulate the actual "sending" of recovery actions and simulate customer responses over time, so you get an end-to-end number.

Features:

- `execute_action(decision_row)` — only runs for `auto_approved`/`modified` decisions; generates a realistic message from a template per action_type/channel, writes to `actions_taken`, increments `customers.contact_count_last_7d`
- `simulate_outcomes()` — a controlled, seeded-random simulator that decides whether each action "worked" based on realistic-ish probabilities (e.g. reminder emails: 25% recover, discount offers: 40% recover, payment link after card update: 55% recover) — **be explicit in your code and demo that this is a simulation standing in for real webhook callbacks**, since you don't have a live payment gateway in the hackathon
- Writes to `outcomes`, updates `risk_events.status`, logs `execution` and `outcome` audit rows

Expected Output: End-to-end pipeline now produces a real recovered-₹ number from synthetic data.

Testing Strategy: Run the full pipeline (detection → diagnosis → policy → executor) on your seed data and sanity-check the final recovery rate isn't absurd (e.g. not 95% or 2%) — tune probabilities if needed so the demo number looks credible (aim for something like 35–55% overall recovery rate, which is realistic and impressive without looking fake).

Phase 5 — Streamlit Dashboard (Day 7–8)

Goal: Build all 7 screens from section 2.4, wired to the live SQLite data.

Features: Multi-page Streamlit app (`st.tabs` or a sidebar `st.selectbox` for navigation is simplest), each screen reading directly from the DB.

Expected Output: A fully clickable, demoable Streamlit app.

Testing Strategy: Click through every page as if you were a first-time merchant. Specifically test the Approvals flow end-to-end (approve one, reject one, confirm audit log reflects both correctly).

Phase 6 — Hinglish / Promise-to-Pay Differentiators (Day 9, optional but high-leverage)

Goal: Add 1–2 "wow" features that are cheap to build but visibly differentiate you from other teams.

Features:

- **Hinglish message generation**: extend the Diagnosis Agent prompt so that when `customer.preferred_language = 'hi-en'`, Gemini generates the recovery message in Hinglish (e.g., "Hi Rahul, aapka ₹1,499 ka payment fail ho gaya tha, yahan click karke turant complete karein"). This is a very on-brand, India-fintech-specific touch.
- **Promise-to-Pay Tracker**: add a `promise_to_pay_date` column to `risk_events` (or a new small table); a simple Streamlit form on the Approvals or Risk Queue page lets the merchant log "customer promised to pay by [date]"; a scheduled check (a button "Check Promises" is fine for MVP) flags any promise past its date as broken and auto-creates a follow-up risk_event.

Expected Output: Two clearly demoable, India-context-specific features.

Testing Strategy: Manually set a customer's `preferred_language` to Hinglish in the DB and confirm the generated message reads naturally, not robotic. Test the broken-promise escalation by backdating a promise date.

Phase 7 — Demo Polish, Numbers, and Story (Day 10)

Goal: Make sure the artifact you show judges tells a tight, confident story in under 4 minutes.

Features / tasks (mostly manual, not code-heavy):

- Tune your synthetic data and simulation probabilities so the final demo run produces a clean, credible headline number (e.g. "₹6,84,200 at risk → ₹2,91,000 recovered → 42.5% recovery rate").

- Reset the DB to a known seed state before recording your demo video so it's reproducible.
- Prepare a 60-second "architecture slide" (can be a static image or even just the Overview + Audit Trail screens) explicitly showing: Detection → Diagnosis (LLM, advisory) → Policy Engine (deterministic, final authority) → Executor → Audit Log.
- Write a one-paragraph README explaining the safety architecture up front — judges reading code on GitHub should see this within 10 seconds of opening the repo.
- Prepare answers to the 3 questions judges will almost certainly ask (see Part 4 below).

Expected Output: A polished repo + a reproducible, impressive demo run.

Testing Strategy: Do a full dry run of your demo, timed. If it's over 4 minutes, cut screens — the Overview, one Diagnosis example, the Policy Engine blocking/modifying something visibly, and the final Reports number are the only four moments you truly need.

PART 4 — JUDGE CRITIQUE (Razorpay Hackathon Judge Mode)

Weaknesses (as currently scoped)

- **All actions are simulated.** There's no real Razorpay webhook integration, no real payment retry via Razorpay's API, no real email/SMS send. Judges will notice this immediately — you must be upfront about it (don't hide it) and frame it as "the recovery logic and safety architecture are production-grade; the last-mile delivery integrations are the fastest thing to bolt on afterward."
- **Outcome simulation is randomized, which is honest but also slightly hand-wavy.** A sophisticated judge may ask "how do you know your recovery rate isn't just your RNG seed?" Have a clear answer: the point being demonstrated is the *decision and safety architecture*, not a real-world recovery rate — be explicit about this distinction rather than implying the % is empirically measured.
- **Single demo merchant, no multi-tenant story.** Fine for MVP, but mention in your pitch that policy_rules and customers are already scoped by merchant conceptually, even if you only demo one.
- **No real retry logic against an actual payment gateway** (e.g., Razorpay's Smart Routing / mandate retry APIs) — this is the single most "real fintech" feature you're missing, and if you have any spare time, even a sandboxed/mocked call to Razorpay's test API for one workflow (e.g., subscription retry) would meaningfully upgrade credibility.

(Note: at-most-once execution / idempotency under concurrent triggers — a common gap in student fintech projects — is now covered by Phase 2.5's DB-level locking and adversarial tests, so this is no longer a weakness relative to more infrastructure-focused submissions.)

Missing Features (nice-to-haves if time allows)

- A real (even if test-mode) Razorpay API integration for at least one action — this alone could be the difference between "good AI project" and "this could ship."
- Rate-limiting / cost tracking on Gemini calls (batch diagnosis calls, cache repeated patterns) — shows production awareness.
- A simple "merchant configures their own policy rules" UI (sliders for discount cap, approval threshold) rather than hardcoded — cheap to add, high perceived value.
- Multi-currency handling (even just displaying it) if you want to gesture at Razorpay's broader (non-India) ambitions.

Risks

- **Scope creep is your biggest risk**, not technical difficulty. Every phase above is intentionally small — resist the urge to add features before Phase 5 (dashboard) is working end-to-end, because a working-but-simple pipeline beats a half-built ambitious one every time in a 10-day window.
- **Gemini API reliability/rate limits during your live demo** — always have a "pre-run" cached demo state as a fallback (run the pipeline once before presenting, keep the DB state, don't re-run live diagnosis on stage unless you're confident).
- **Over-explaining the architecture and running out of demo time** — practice the 4-minute cut ruthlessly.

Ways to Stand Out From Other Teams

- Most teams in this track will build "detect problem → LLM writes a message → send it." Very few will build a genuine **deterministic policy layer that can override the LLM**, with a visible audit trail. Lead your pitch with a moment where the policy engine *disagrees with and overrides* the AI — that single demo beat (LLM says "offer 25% off," Policy Engine says "capped at 10%, logged") is worth more than any chart.
- Explicitly using the phrase **"the LLM recommends, it never decides"** in your pitch, tied to a live example, directly mirrors language a payments-company judge wants to hear.
- The **Promise-to-Pay tracker** and **Hinglish messaging** are both cheap, authentic, India-context features most CS-student teams won't think to add — they read as product sense, not just engineering.
- The **provider-agnostic LLM layer with automatic fallback** (Gemini ↔ Groq) is a small addition that signals a level of production thinking ("what if my vendor has an outage during the panel demo?") that almost no student submission will have considered — mention it explicitly in your pitch as a reliability decision, not just a cost workaround.

- **Live-demoing the idempotency lock** (double-click "Run Recovery" on the same event, show the second click cleanly rejected, not crashed or duplicated) is a concrete, interactive proof of a safety property — more convincing in a 5-minute panel demo than any slide claiming the property exists.
- Show the **stopping rules working**, not just the happy path — deliberately demo a blocked/expired case (max attempts reached, customer opted out) since "knowing when to stop" is explicitly called out in the brief and almost nobody will demo it.

What Would Make This Look Like a Production Fintech Product, Not a Student Project

- A visible, merchant-editable policy configuration screen (even a simple one) — turns "hardcoded rules" into "a governance system."
- Idempotency handling made explicit (e.g., a note/check that re-running detection doesn't double-count the same risk_event) — small thing, but signals real engineering discipline.
- A one-line "compliance note" on the channel-hours rule (9am–8pm) referencing that this mirrors real DLT/TRAI-style SMS/WhatsApp regulations in India — shows you understand the *why*, not just the *what*.
- Clean separation of concerns in the codebase (exactly as scoped in Phase 0's folder structure) — when a judge skims your GitHub repo, the file structure itself should communicate the architecture before they read a line of code.
- A crisp, non-hyped README stating what's real vs simulated — paradoxically, being explicit and honest about the simulation boundary reads as *more* credible to experienced judges than pretending everything is live.

One last piece of advice: build phases 0–4 (the pipeline) completely before touching Streamlit. It is much easier to debug detection/diagnosis/policy logic with print statements and direct DB queries than through a UI. The dashboard is the last thing you build, not the thing you build alongside everything else.